Second-Chance Skills 장문 해설서

작성일: 2026-10-12. 이 문서는 첫 주 연구 설계의 이해와 실행 준비를 위한 해설서다. 실제 모델의 성능·회복 효과·재발 억제 효과는 NR(미실측)이다. 프로토콜은 잠기지 않았고 사전등록은 완료되지 않았다. 지금 있는 것은 오프라인 합성 검증 도구, 준비된 공개 과제의 로컬 변환, 그리고 앞으로 실행할 비교 설계다. 계산 예시는 모두 교육용 가상 예시이며 관측된 모델 결과가 아니다. 출처 해시는 같은 자료를 읽었는지 확인하는 식별자이고 학술적 정확성이나 신규성 인증이 아니다.

1. 연구 질문: 정답을 바로잡아도 에이전트의 현재 상태는 남는다

Second-Chance의 출발점은 평가가 후보의 점수만 기록하는 행위가 아니라 이후 행동을 만드는 사건이라는 점이다. 에이전트가 새 스킬을 작성하고 평가기에 제출한다고 하자. 평가기가 틀린 기준으로 점수를 매기면, 올바른 후보가 탈락하고 잘못된 후보가 현재 스킬로 남을 수 있다. 탈락 이유를 반성문이나 교훈으로 저장하는 시스템이라면 "그 접근을 다시 쓰지 말라"는 지침까지 남는다. 평가기를 교정하는 순간 그 지침의 근거는 약해지지만, 지침의 문장이 자동으로 삭제되거나 현재 스킬이 자동으로 교체되지는 않는다.

이때 과거 결과표를 다시 계산해 좋은 점수를 얻는 것과 에이전트가 지금 올바른 답을 내는 것은 다르다. 전자는 같은 답안에 새로운 점수를 붙이는 일이다. 후자는 현재 활성 스킬, 실제로 검색되어 들어오는 교훈, 도구 상태와 새 입력을 포함한 실행이다. 과거 탈락 후보가 재채점에서 성공해도 현재 상태가 그 후보를 사용하지 않으면 회복은 일어나지 않을 수 있다. 반대로 탈락 후보를 그대로 복원하지 않고 현재 스킬을 새로 고쳐 같은 능력을 되찾을 수도 있다. 따라서 논문은 후보의 명예 회복이 아니라 현재 능력의 회복을 주된 결과로 본다.

연구 질문을 더 구체적으로 쓰면 다음과 같다. 과제 요구를 고정한 채 잘못된 평가기가 만든 선택 이력을 보존하고, 독립적으로 교정을 확인한 다음, 같은 보관 후보와 같은 예산을 받은 정책들을 비교할 때, 선택의 근거와 부정적 교훈까지 재검토하는 정책이 단순한 평가기 교체·현재 승자 수정·보관 후보 재생보다 현재 상태의 성능을 더 잘 회복하는가? 이 질문은 실제 선택 이력이 있을 때만 검증할 수 있다. 평가 오류 목록이나 좋은 후보 몇 개를 미리 만들어 두는 것만으로는 그 현상이 발생했다고 할 수 없다.

여기서 "오염"은 모델 가중치가 영구적으로 변했다는 뜻이 아니다. 기본 연구 대상은 버전이 있는 스킬 파일, 현재 후보를 가리키는 포인터, 외부 메모리의 교훈, 그리고 이들이 입력으로 노출된 기록이다. 잘못된 평가의 영향이 이 외부 상태를 통해 이어졌다는 가설을 다룬다. 가중치 수준의 학습이나 삭제 효과는 별도의 개입과 검증이 필요하다. 현재 도구는 이런 학습을 수행하지 않는다.

좋은 결과만 모아 연구 질문을 성립시켜서는 안 된다. 잘못된 평가를 실제로 겪었지만 유용한 탈락 후보가 없었던 이력, 교훈이 작성되지 않은 이력, 교정해도 승자 순위가 바뀌지 않는 이력도 보존해야 한다. 그런 사례가 많으면 제안 방법의 적용 범위가 좁다는 결론이 나올 수 있다. 연구의 첫 단계는 유리한 사례를 만드는 것보다 현상이 얼마나 자주 관측 가능한지 확인하는 것이다.

2. 고정된 요구 G*와 평가기 버전: 무엇이 교정이고 무엇이 새 문제인가

G*는 실험을 위해 고정한 운영상 요구와 독립 검토된 채점 기준을 뜻한다. 별표가 모든 과학적 진리를 보장한다는 의미는 아니다. "섭씨를 켈빈으로 변환할 때 273.15를 더한다", "동점은 식별자 순으로 정렬한다", "NULL을 평균의 분모에서 제외한다"처럼 명시적인 요구를 정할 수 있다. 그 요구를 코드로 구현한 평가기 V_old에 오류가 있으면, 같은 요구를 올바르게 구현한 V_new로 교정한다. 요구 문서와 허용 정답은 그대로이고 구현의 판단만 바뀌는 것이 기본 계약이다.

사용자가 처음에는 입력 순서로 동점을 처리하라고 했다가 나중에 이름순으로 바꿨다면 이는 평가기 오류 교정이 아니다. 처음 요구에 맞던 후보가 새 요구에 맞지 않는 것으로 바뀐 상황이다. 새 분포에 적응하는 연구로는 의미가 있지만 Second-Chance의 기본 인과 질문과 섞으면 안 된다. 요구 변경에서는 과거의 부정적 교훈이 당시에는 타당했을 수 있기 때문이다. 이 연구에서는 과거 판단의 근거가 처음부터 같은 요구에 맞지 않았다는 점을 분리한다.

허용 가능한 대안 정답도 교정 전에 정해야 한다. 어떤 분류 과제가 "supported"와 사전에 합의한 동의어를 모두 허용한다고 문서에 적었는데 평가기가 한 문자열만 받았다면, 다른 허용 답안을 거절한 것은 평가 오류다. 반대로 실험 결과를 보고 탈락 후보의 표현만 뒤늦게 허용 목록에 넣으면 정답 기준이 정책에 맞춰 변할 위험이 있다. 표현 정규화, 숫자 허용 오차, 빈 입력의 반환 규칙, 증거 문장 대안은 교정 검토에서 기록하고 고정해야 한다.

독립 교정은 평가기 작성자가 자신이 틀렸다고 인정하는 것만을 뜻하지 않는다. 정책 코드와 후보 실행 코드에서 기대 답을 가져오지 않은 별도 채점 구현, 경계값과 대안 답안에 대한 반례, 필요할 경우 사람의 판정이 있어야 한다. 정책이 원하는 답을 내고 그 답을 평가기가 그대로 정답으로 정의하면 독립성은 사라진다. 기본 도구의 코드 모듈 분리는 이 문제를 줄이는 장치지만, 운영체제 수준의 비밀 파일 격리까지 제공하는 것은 아니다.

과학적 주장 검증에서는 G*의 한계를 더욱 조심해야 한다. SciFact의 annotation은 주어진 논문 초록과 사람의 근거 표시에 대한 데이터셋 정답이다. 이를 모든 시점과 모든 논문에 대한 과학적 사실로 확대하지 않는다. 특정 초록에 주장을 지지하는 근거가 표시되지 않았다고 해서 주장이 세계 전체에서 거짓이라는 뜻은 아니다. 불명확한 사례를 억지로 단일 정답으로 만드는 대신 판정 보류나 별도 사람 검토 대상으로 남길 수 있어야 한다.

교정 사건은 이전 평가기 해시, 교정 평가기 해시, 고정 요구 해시, 문제가 된 규칙, 반례 입력, 허용 답안 및 검토 상태와 연결한다. 이것은 교정이 실제로 무엇을 바꿨는지 추적하기 위한 자료다. 해시가 맞는다고 교정의 내용이 옳다고 인증되지는 않는다. 같은 바이트를 확인하는 무결성과 채점 기준의 타당성은 두 개의 검증 단계로 유지한다.

3. 가상 사례와 순위 역전 식: 재검토 신호와 회복을 구별하기

다음 사례의 후보·입력·숫자는 모두 교육용이다. 실험에서 관측된 모델의 결과가 아니다. 공개 요구가 섭씨에 273.15를 더하라는 것인데, 잘못된 평가기는 273을 더한 답안을 정답으로 채점한다고 하자. 기존 후보 w는 평가기 오류에 맞춘 변환을 한다. 새 후보 c는 공개 요구에 맞는 273.15를 사용한다. 두 후보가 같은 입력 묶음에서 답을 냈을 때 V_old는 w를 선택하고 c를 탈락시킨다.

탈락 사유를 외부 교훈으로 저장하면서 "소수점 오프셋은 불필요한 변형이므로 정수 오프셋을 유지하라"는 문장을 만들었다고 하자. 이후 현재 스킬은 다른 작업도 개선했지만 이 교훈은 계속 활성화될 수 있다. V_new를 교정하고 c의 과거 답안을 다시 채점하면 성공할 수 있다. 그러나 현재 스킬이 여전히 w에서 파생되어 있고 교훈도 유지된다면, 새 입력에는 다시 273을 더할 수 있다. 이것이 재채점 성공과 현재 회복의 차이를 보여 주는 개념 사례다.

순위 역전을 판단할 때는 같은 저장 답안을 두 평가기로 채점해야 한다. s_old(c)는 c가 과거에 만든 고정된 답안 묶음의 옛 점수, s_new(c)는 그 동일한 답안의 교정 점수다. 옛 승자 w와 탈락 후보 c 사이의 여유를 m으로 두고 각 후보의 점수 변화량을 Δ로 두면 다음 관계가 성립한다.

$$m = s_{\text{old}}(w) - s_{\text{old}}(c) \geq 0$$

$$\Delta(c) = s_{\text{new}}(c) - s_{\text{old}}(c)$$

$$s_{\text{new}}(c) - s_{\text{new}}(w) = \Delta(c) - \Delta(w) - m$$

이 식은 저자가 제안하는 단순한 대수 항등식이다. 마지막 줄은 새 점수 차이를 옛 점수 차이와 각 후보의 교정량으로 나누어 쓴 것이다. 새로운 비교 기준이 "c가 w보다 δ를 초과하여 좋아야 한다"라면, Δ(c)−Δ(w)>m+δ가 그 고정 답안 쌍에서 엄격한 순위 역전의 필요충분조건이다. 관측 자료를 대신하는 정리는 아니며, 이 식 하나로 현재 행동의 개선을 증명할 수 없다.

교육용 수치를 넣어 보자. s_old(w)=0.80, s_old(c)=0.60이면 m=0.20이다. 교정 후 각각 0.55와 0.90이 되면 Δ(w)=−0.25, Δ(c)=0.30이다. 상대 교정량은 0.55이고, 새 점수 차이는 0.55−0.20=0.35다. δ=0.10으로 정했다면 0.55>0.30이므로 역전 조건을 만족한다. 이 숫자는 식을 확인하기 위한 가상 점수이고 이 프로젝트의 성능표에 들어가지 않는다.

반례도 필요하다. c가 0.60에서 0.80으로 좋아지고 w가 0.80에서 0.85로 좋아졌다면 상대 교정량은 0.15에 그친다. m=0.20을 넘지 못해 순위는 바뀌지 않는다. 평가기 교정이 있었다는 사실만으로 모든 탈락 후보를 다시 채택하면 안 되는 이유다. 동점이 생겼다면 사전에 정한 동점 처리 규칙을 유지하고, 당시 실제 탈락 여부도 기록 그대로 보존해야 한다.

순위가 뒤집힌 후보가 새 실행에서 실패할 수도 있다. 과거 답안은 우연히 맞았거나 당시의 도구 상태에 의존했을 수 있고, 현재 스킬과 합쳤을 때 후속 기능이 손상될 수도 있다. 반대로 과거 순위가 그대로인 후보가 새 과제에서는 유용할 수도 있다. 순위 역전은 재검토 우선순위를 위한 신호이며, 독립 자격화와 현재 상태 평가를 대체하지 않는다. 순위 역전 후보만 선택해 연구 전체를 구성하면 다른 유용한 탈락 후보를 놓치는 선택 편향도 생긴다.

4. 내용·선택·교훈 계보: 같은 파일 해시로는 판단 이력이 보이지 않는다

내용 계보는 어떤 스킬이나 문맥이 새 파일 또는 답안의 작성에 실제 입력으로 사용되었는지 보여 준다. 후보 A를 고쳐 B를 만들었다면 A의 버전에서 B의 작성 사건으로 이어지는 내용 연결이 있다. B

의 파일 해시와 부모 해시를 남기면 파일의 유래는 추적할 수 있다. 그러나 A가 왜 선택되었는지, 다른 후보 C가 왜 탈락했는지, 그 탈락이 어떤 교훈으로 이어졌는지는 파일 유래만으로 알 수 없다.

선택 연결은 점수 비교 사건에서 현재 승자 포인터로 이어지는 관계다. 후보 두 개의 ID, 당시 평가기, 답안과 점수, 허용 여유, 동점 처리, 선택 결과가 함께 있어야 한다. 후보가 내용 작성에 사용되지는 않았어도 "이 후보는 실패했다"는 비교 정보가 다음 선택에 영향을 줄 수 있다. 탈락 후보는 내용 계보에서 사라지기 쉽지만 의사결정 계보에서는 중요한 노드다.

교훈 연결은 탈락 판단과 평가 근거에서 외부 교훈 버전으로 이어진다. 교훈이 저장되었다는 사실만으로 매 실행에 영향을 주었다고 단정할 수는 없다. 어떤 문맥에서 검색되었고 모델 입력에 실제로 포함되었는지 노출 사건까지 기록해야 한다. 내용이 동일한 교훈도 다른 실행에서 서로 다른 문맥과 함께 노출될 수 있으므로 각각의 사건을 구분한다. "파일이 있었다"와 "그 문장을 보고 행동했다"의 차이가 원인 분석에서 중요하다.

간단한 개념 흐름을 쓰면 옛 답안→V_old의 채점 근거→탈락 결정→교훈 v1→후속 문맥 노출→현재 답안이다. 별도로 옛 채점 근거→승자 선택→활성 스킬 포인터→현재 답안의 흐름도 있다. 하나는 현재 선택된 스킬을 통해, 다른 하나는 부정적 지침을 통해 이어진다. 두 흐름이 같은 최종 답안에서 만나도 개입 지점은 다르다. 승자를 바꾸기만 하는 정책과 교훈을 수정하기만 하는 정책을 분리하는 이유다.

계보는 버전이 펼쳐진 시간순 그래프로 표현한다. 스킬 A가 나중에 다시 선택되었다고 해서 예전 A 노드로 역방향 연결을 만들지 않는다. 같은 파일 내용이라도 새 선택 사건과 새 상태 버전을 만든다. 그래야 과거 사건이 미래 사건의 근거가 되는 방향을 유지하고 순환 없는 그래프에 대한 분석을 적용할 수 있다. 파일 이름 하나를 영구적인 노드로 취급하면 반복 선택 때문에 시간과 원인 방향이 섞일 수 있다.

부정적 교훈을 고칠 때도 이전 내용을 덮어쓰지 않는다. v1이 어떤 근거를 사용했는지 남겨 두고, v2에서 그 근거가 교정되었음을 기록한다. 새 버전은 "모든 정수 오프셋은 나쁘다"처럼 반대 방향으로 과잉 일반화하기보다, 잘못된 채점 규칙과 영향을 받은 범위에 한정해야 한다. 다른 유효한 근거를 함께 가진 교훈은 일부 범위만 수정하거나 보류할 수 있다. 복구는 기억을 무조건 지우는 행위보다 지원 근거의 유효성을 갱신하는 행위에 가깝다.

이 기록으로 확정할 수 있는 것은 노출과 지원 관계다. 기록된 교훈을 본 뒤 잘못된 답이 나왔다고 해서 그 교훈이 유일한 원인이라는 뜻은 아니다. 모델의 사전 지식, 문맥의 다른 지침, 도구 결과가 같은 오류를 만들 수 있다. 따라서 계보는 어떤 개입을 시험할지 정하는 지도이고, 실제 원인 효과는 대조군과 독립 실행으로 확인한다. 현재 v3는 이 구조의 합성 연결을 제공하며 자연스러운 장기 계보 수집은 미구현 상태다.

5. 재검토 경로의 충분조건: 어디까지 보장하고 어디서 실패하는가

교정으로 무효화된 채점 근거나 판정 노드 집합을 D, 지금 살아 있는 스킬 포인터와 외부 교훈 노드 집합을 T라고 하자. Desc(D)는 D에서 앞으로 도달할 수 있는 모든 기록 노드, Anc(T)는 T로 도달할 수 있는 모든 기록 노드다. 여기서는 시작 노드 자체도 포함한다. 두 집합의 교집합 R은 오류 근거에서 현재 상태로 이어지는 기록 경로 위의 노드를 모은다.

$$R = \mathrm{Desc}(D) \cap \mathrm{Anc}(T)$$

왜 그런지 양방향으로 확인할 수 있다. D에서 T로 가는 어떤 경로 위의 노드는 D에서 도달 가능하고 T로 도달 가능하므로 교집합에 속한다. 반대로 교집합의 노드는 D에서 그 노드로 오는 경로와 그 노드에서 T로 가는 경로를 가진다. 버전이 펼쳐진 순환 없는 그래프에서는 이를 이어 D에서 T로 가는 경로를 만들 수 있다. 따라서 R은 기록된 D→T 경로의 노드 집합과 같다. 이 주장은 실제로 기록한 그래프에 대한 저자 분석이다.

의미 있는 충분조건은 "모든 관련 의존성을 빠짐없이 기록했고 교정의 시작점을 올바르게 찾았다면, R을 검토하여 기록된 오류→현재 상태 경로를 빠짐없이 감사할 수 있다"다. 행동 회복까지 보장하지 않는다. R의 모든 노드를 알아도 실제 스킬을 어떻게 고쳐야 하는지 모를 수 있고, 후보가 사라졌거나 예산이 부족할 수도 있다. 교정한 판단의 영향이 모델 가중치나 기록하지 않은 외부 도구에 남아 있다면 이 그래프 밖에서 효과가 계속될 수 있다.

"최소 재검토 집합"이라는 표현에는 특히 주의해야 한다. R은 기록 경로를 구성하는 모든 노드의 집합이지만, 최소 비용으로 효과를 끊는 개입 집합과 같지는 않다. 하나의 교훈 노드만 수정해 여러 경로를 차단할 수 있는 반면, 한 노드의 파일이 여러 기능을 담당해서 안전하게 바꾸려면 더 넓은 검토

가 필요할 수 있다. 정점 절단, 근거 감사, 행동 수정은 서로 다른 최적화 문제다. 현재 설계는 최소 비용 최적화 정리를 주장하지 않는다.

가상 실패 사례를 보자. 잘못된 채점에서 만들어진 교훈이 중간 단계에서 요약 메모로 복사되었는데 복사 연결을 기록하지 않았다면, R에는 원래 교훈만 잡히고 요약 메모는 빠질 수 있다. 원래 교훈을 고쳐도 모델은 요약 메모를 보고 같은 금지 규칙을 따를 수 있다. 이것은 R의 대수 정의가 틀렸다는 반례가 아니라, 실제 의존성의 완전성이라는 가정이 성립하지 않은 사례다. 삭제 연결 대조군은 이런 취약성을 드러내는 시험이다.

다른 반례는 잘못된 시작점이다. 평가기에서 단위 변환 규칙을 고쳤는데 실제 문제는 동점 판정에도 있었다고 하자. 단위 변환 근거만 D에 넣으면 동점에 연결된 잘못된 선택은 대상에서 빠진다. 독립 교정 검토와 그래프 추적을 따로 두는 이유다. 계보 추적을 완벽하게 수행해도 교정 대상 자체를 잘못 찾으면 현재 문제를 충분히 설명하지 못한다.

R 밖의 탈락 후보를 모두 무시해서도 안 된다. 현재 상태로 이어지는 기록 경로가 없는 후보가 새 독립 과제에서 유용할 수 있다. 이런 후보의 발견은 기록된 영향 경로를 수선하는 문제와 보관함에서 새 능력을 찾는 문제의 차이다. 따라서 결정 계보 정책은 R에 연결된 우선순위를 쓰되 같은 보관 후보 접근 권한을 가진 무작위·여유 기반 재생과 비교한다. 완전한 보관함을 숨은 정답으로 탐색하는 오라클은 참고 진단이며 주 비교군으로 섞지 않는다.

6. 왜 단순 재라벨링·후견 최적화·재튜닝·검색과 다른가

재라벨링은 답안이나 학습 예제에 붙은 표지를 바꾸는 일이다. 이 연구에서도 같은 과거 답안을 V_new로 채점하는 단계는 필요하다. 하지만 에이전트가 잘못된 점수를 받아 어느 후보를 버렸고 어떤 교훈을 저장했는지까지 바뀌는 것은 아니다. 표만 새로 고친 뒤 현재 상태를 그대로 실행하는 대조군을 두면 재라벨링만으로 충분한 경우와 부족한 경우를 관찰할 수 있다.

후견 최적화는 결과를 모두 본 뒤 좋은 후보를 고르는 넓은 접근을 말한다. 최종 평가 정답을 본 오라클이 모든 후보를 비교하면 회복 가능한 상한을 진단하는 데 유용하다. 그러나 실제 정책은 제한된 개발 피드백과 예산만 받아야 한다. 오라클의 정보로 좋은 후보를 골라 놓고 결정 계보 정책이 잘 골랐다고 보고하면 선택자의 성능을 과장한다. 숨은 최종 정답에 접근하는 정책과 접근하지 않는 정책은 같은 비교가 아니다.

현재 승자 재튜닝은 문제를 겪는 활성 스킬만 고치는 방법이다. 잘못된 교훈을 보지 않아도 개발 피드백이 충분하면 새로운 패치가 정답을 회복할 수 있다. 그 방법이 싸고 성능도 같다면 보관 이력 복구를 추가할 실용적 근거는 약해진다. 따라서 현재 승자 수정은 약한 대조군으로 취급하지 않는다. 잘 작성된 현재 수정 정책이 탈락 후보의 재생보다 나을 가능성도 연구에서 열린 결론으로 둔다.

보관함 검색은 유용한 과거 후보를 다시 찾는 방법이다. 무작위 순서, 과거 점수 여유, 문서 유사도, 내용 계보로 후보를 선택할 수 있다. 결정 계보는 여기에 "지금 교정된 판단이 이 탈락과 어떤 교훈을 지원했는가"라는 관계를 추가한다. 이 관계가 후보 탐색을 줄이거나 잘못된 금지 규칙을 수정하는 데 도움이 되는지 확인한다. 단순 검색도 교정된 개발 점수를 받으면 충분히 잘할 수 있으므로 같은 후보와 정보로 비교해야 한다.

부정적 교훈의 수정이 특별한 이유는 선택된 스킬을 바꾼 다음에도 검색되는 지침이 미래 행동을 제한할 수 있기 때문이다. 그렇다고 모든 교훈을 고치면 더 좋다는 결론은 아니다. 과거 오류를 바탕으로 작성되었어도 우연히 유용한 경고를 담았거나, 다른 유효한 사례와 결합되어 있을 수 있다. 근거를 분리하지 않고 지우면 다른 능력이 손상된다. 교훈의 범위를 좁혀 버전으로 남기는 방식과 교훈을 유지하는 방식의 차이를 독립 실행으로 확인한다.

선택 복구와 교훈 복구를 동시에 한 정책이 좋아졌다면 그 결과만으로 어느 구성요소가 작용했는지 알 수 없다. 같은 선택 과정을 수행하되 교훈만 고정한 군, 현재 스킬은 그대로 두고 교훈만 수정한 군, 둘 다 수정한 군이 필요하다. 이는 신규성 주장보다 먼저 식별 가능한 실험을 만들기 위한 설계다. 구성요소 효과가 확인되지 않으면 "결정 이력 복구"라는 넓은 이름으로 결과를 포장하지 않는다.

현재 연구는 처음 만든 완전한 복구 에이전트라고 주장하지 않는다. 공개 구현의 기능과 겹치는 부분은 대조 방법으로 검토하고, 교정 사건 이후의 판단·교훈 수선과 현재 상태 평가라는 좁은 질문을 남긴다. 실제 구현 동등성, 다른 연구의 세부 절차, 더 넓은 문헌의 우선성은 아직 미확정이다. 이 경계를 지켜야 첫 주 설계의 기여가 실행 가능한 검증 질문으로 이어진다.

7. 선행연구를 읽는 기준: 기능의 존재와 이번 질문의 답은 다르다

공식 원문과 저장소를 읽을 때는 무엇이 이미 있는지와 무엇을 아직 비교하지 않았는지를 구분한다. SkillOpt의 공식 초록은 거절된 수정의 버퍼와 제한된 최적화·보류 평가를 설명한다. 따라서 "탈락 후

보를 저장한다"는 사실은 이 연구의 새로운 기여가 아니다. 공식 SkillOpt-Sleep 문서는 재생, 반성, 기억 갱신과 검증 절차까지 설명한다. 이 검토 때문에 보관함 재생이나 교훈 갱신 자체를 독창성의 근거로 삼을 수 없다. SkillOpt v2 (https://arxiv.org/abs/2605.23904v2), SkillOpt-Sleep 공식 문서 (https://github.com/microsoft/SkillOpt/blob/main/docs/sleep/README.md).

Self-Refine은 답안의 자기 피드백과 반복 수정을, Reflexion은 언어적 반성과 에피소드 기억을 통해 다음 시도를 바꾸는 구성을 다룬다. Generative Agents는 경험 기억의 검색과 반성을 다루며 공식 HTML에서 관련성·최근성·중요도에 기반한 검색을 확인했다. 이들은 답안 수정이나 기억 선택의 실질적인 인접 방법이다. 이번 실험에서 각각의 대조 구현을 아직 만들거나 성능을 비교하지 않았다. Self-Refine (https://arxiv.org/abs/2303.17651), Reflexion (https://arxiv.org/abs/2303.11366), Generative Agents (https://arxiv.org/abs/2304.03442v2).

라벨 품질을 다루는 Confident Learning과 평가기 편향을 다루는 LLM-as-a-Judge 연구도 중요하다. 전자는 라벨 오류와 불확실성을 다루고, 후자의 공식 초록은 위치·장황함·자기 선호 등 판정 편향을 보고한다. 이런 보고는 교정의 필요성을 뒷받침하는 배경이지 이 과제에서 같은 크기의 오류가 발생했다는 증거가 아니다. 본문은 선행연구의 성능 숫자를 가져오지 않는다. Confident Learning (https://arxiv.org/abs/1911.00068v6), Judging LLM-as-a-Judge (https://arxiv.org/abs/2306.05685).

여기에서 남는 질문은 고정된 요구에 대한 교정이 일어났을 때, 잘못된 비교에 의해 만든 승자와 교훈의 지원 근거를 어떻게 다루며, 그 처리가 독립적인 현재 실행의 회복을 바꾸는가다. 기존 방법이 이 질문을 이미 완전히 해결했는지 판단하려면 코드와 실험 절차를 버전 고정하여 더 깊게 비교해야 한다. 공식 README에서 기능을 확인한 것은 출발점이고, 구현이 완전히 같은 문제를 다룬다는 인증이 아니다. 반대로 원문에 구체적인 문장이 없다는 이유만으로 그 방법이 해당 기능을 할 수 없다고 단정하지 않는다.

공정한 비교의 후보는 단지 이름이 다른 방법이 아니라 같은 작업을 수행할 수 있는 구체적인 구현이어야 한다. 예를 들어 보관함 기반 방법에 후보 전부를 읽을 예산을 주지 않고 결정 계보 방법에 미리 고른 후보만 주면 비교가 틀어진다. 반성 기반 방법에 교정 알림을 숨기면 역시 불공정하다. 최종 비교 전에는 어떤 정보가 입력으로 들어가고, 어느 시점에 수정과 평가가 가능하고, 비용은 어디에 붙는지를 같은 계약으로 맞춰야 한다.

첫 주의 문헌 검토는 제한된 공식 초록·서지·HTML 일부·공식 저장소 문서 검토다. 임의 PDF를 수집하지 않았고 외부 코드를 실행하지 않았다. SkillOpt-Sleep 문서는 mutable main 상태로 읽었으므로 실행 대조군으로 쓸 때는 커밋을 고정하고 다시 확인해야 한다. SOURCES.json은 확인 범위와 미확인 범위를 남긴다. "공식 출처를 찾았다"를 "전체 방법의 동일성·차이·최초성을 검증했다"로 바꾸지 않는 것이 이번 설계의 문헌 원칙이다.

8. 실제 파이프라인: 오류 레코드에서 관측된 선택 이력으로 이동하기

완성된 실험의 첫 입력은 오류 레코드가 아니라 고정한 과제 묶음, 실제 모델 설정, 초깃값 스킬과 진화 절차다. 공개 과제와 허용 정답 계약을 고정하고, 평가 전용 정답을 정책 문맥에서 분리한다. 모델·입력 전송 범위·요청과 토큰·비용 한도가 아직 정해지지 않은 상태에서는 실제 호출을 시작하지 않는다. 이 작업은 오프라인 설계 작성이며 새 모델을 실행하지 않았다.

다음에는 후보를 실제로 생성한다. 초기 스킬과 같은 개발 과제를 받은 모델이 제한된 수정 후보를 작성하고, 원본 파일·문맥·도구 결과·응답·모델 설정·시각·사용 자원을 기록한다. V_old로 그 후보의 답을 채점한 뒤 실제 선택 규칙에 따라 수용 또는 거절한다. 결과가 미리 정해진 후보를 "모델이 탈락시켰다"고 쓰지 않는다. 출력 실패나 스킬 생성 보류도 후보 기록에 남긴다.

잘못된 교훈의 존재 역시 모델이 실제로 작성하고 노출한 기록으로 확인해야 한다. 어떤 진화 시스템이 거절을 텍스트 교훈으로 저장하지 않는다면, 그 이력은 교훈 효과의 대상이 아닐 수 있다. 다른 경우에는 교훈을 썼지만 이후 문맥에 한 번도 넣지 않아 영향 경로가 형성되지 않을 수 있다. 교훈을 강제로 만들어 항상 오류를 재발시키는 합성 도구는 측정 검증에는 쓸 수 있어도 자연 현상의 빈도를 추정하는 자료가 아니다.

정해 둔 진화 시점까지 이력을 수집한 뒤 평가기를 교정한다. 교정 시점을 좋은 탈락 후보가 처음 등장하는 순간으로 골라서는 안 된다. 예를 들어 고정한 반복 횟수나 비용 지점에서 교정한다는 규칙을 먼저 정하고, 그 지점의 현재 스킬·활성 교훈·보관 후보·점수·노출 이력을 동결한다. 정책 배정 전에 이 공통 상태를 고정해야 복구 정책에 유리한 이력을 뒤늦게 선택하는 위험을 줄인다.

각 복구 정책은 동결 상태의 별도 사본에서 시작한다. 같은 원본 후보와 교정 알림에 접근하고, 같은 개발 피드백 권한과 비용 상한을 받는다. 다른 정책의 답안이나 교훈 수정이 공유 메모리에 반영되지 않도록 분리해야 한다. 이런 실행 격리는 현재 오프라인 패킷 내보내기만으로 보장되지 않으며 실제

runner에서 구현해야 한다. 공유 도구 상태가 필요하다면 읽기 전용 스냅샷 또는 정책별 분리를 명시한다.

복구 과정에서는 먼저 저장 답을 재채점하고, 필요한 후보를 새로 실행한 뒤, 교정 개발 기준을 통과한 수정만 현재 상태에 통합한다. 실패한 후보를 현재 포인터로 올리거나 예산 부족으로 평가하지 못한 후보를 성공으로 처리하지 않는다. 마지막으로 독립 채점자는 정책이 볼 수 없는 새 관측을 사용해 활성 상태를 평가한다. 정책 선택에 쓴 개발 성공과 최종 성능은 분리한다.

그 뒤 별도 지속 진화 구간에서 다시 후보를 만들고 선택하도록 할 수 있다. 여기서 오류가 재발하는지 보려면 초기 회복 자격화와 다른 과제를 사용하고, 교훈의 실제 노출 여부를 기록해야 한다. 한 번 회복한 뒤 다시 실패한 사례와 처음부터 회복하지 못한 사례를 구분한다. 전자는 재발의 조건부 분모에 들어갈 수 있지만 후자는 현재 회복의 실패이며, 두 현상을 같은 이름으로 합치지 않는다.

현재 준비 상태는 이 전체 흐름의 앞부분에 있다. v3는 파일·계보·예산·지표의 합성 연결을 시험하고, v4는 실제 공개 과제의 변환과 오류 오버레이를 제공한다. 실제 모델의 거절 결정과 교훈을 생성하는 진화 runner, 상태 활성화 증거, 독립 감사의 운영체제 격리는 아직 필요하다. 해설서의 파이프라인은 실행 계획이며 이미 끝난 절차로 서술하지 않는다.

9. SciFact를 어떻게 이식하는가: 531개 과제와 44개 오류의 정확한 의미

로컬 변환의 이름은 SciFact-CitedPair-RFv4다. SciFact 원 과제의 전체 문헌 검색을 재현하는 것이 아니라, 하나의 주장과 선택된 인용 문서 초록을 주고 해당 문서에서 지지·반박·불충분을 판단하게 하는 인용쌍 과제다. 원 데이터의 문서·주장·근거 문장 구조를 활용하지만 공식 리더보드 점수를 재현한다고 주장하지 않는다. 원 연구의 정확한 제목과 공식 출처는 Fact or Fiction: Verifying Scientific Claims (https://aclanthology.org/2020.emnlp-main.609/)다.

v4 준비 기록의 원 코퍼스는 5,183개 문서, 공식 train은 809개 주장, 공식 dev는 300개 주장이다. 모든 인용 문서 연결과 정규화된 동일 주장 연결을 통해 성분을 만들고, 공식 dev와 연결된 train 278개를 제외했다. 남은 적격 과제는 531개이며 연결 성분은 325개다. 최종 로컬 분할은 train 306개/193성분, development 100개/62성분, test 125개/70성분이다. 과제 수와 성분 수는 둘 다 보고한다.

이 숫자들은 "독립 실험이 531회 있었다"라는 뜻이 아니다. 하나의 인용 성분 안에 비슷한 근거를 공유하는 과제가 있을 수 있고, 여러 정책이 같은 진화 이력에서 갈라질 수도 있다. 반복 생성까지 포함하면 더 많은 응답이 생기지만 독립 정보량이 그 수만큼 늘지는 않는다. 과제 수는 채점 분모를 설명하고, 성분 수는 의존성과 분할 단위를 설명하며, 진화 이력 수는 복구 정책 비교의 표본 단위를 설명한다. 현재 실제 후보 이력과 교정 이력은 각각 0개다.

문서 선택은 정답을 보기 전에 인용 문서의 SHA 기반 규칙으로 정했다. 연결 성분 SHA의 modulo 10 규칙은 train/development/test를 0–5/6–7/8–9로 배정한다. 클래스 라벨로 좋은 예제를 고르거나 좋은 성능이 나는 분할을 다시 찾지 않았다는 준비 계약이다. 이 규칙이 의미적 중복을 모두 제거한다는 보장은 없다. 표면적 동일 주장과 인용 관계 밖의 유사 내용은 남을 수 있으므로 별도의 감도 분석이 필요하다.

44개는 train의 지지·반박 라벨 일부를 결정적으로 맞바꾼 오류 오버레이 레코드 수다. 원 정답은 그대로 보존하고 V_old가 해당 과제를 잘못 판단하도록 구성한다. V_new는 독립 검토한 원 기준에 맞는 판단을 사용한다. 이 오버레이는 원 데이터셋이 공식적으로 인정한 오류 이력도 아니고, 실제 평가기 운영에서 발견된 44건의 버그도 아니다. 특히 44개의 수정 레코드만 보고 44개의 모델 거절·복구 이력이 있다고 셀 수 없다.

실제 과제로 전환하려면 공개 claim, title, sentences를 정책에 주고 모델이 라벨과 근거 문장 식별자를 출력하게 한다. V_old의 오버레이 아래 실제 후보 선택과 교훈 생성이 발생해야 한다. 교정 후 개발과 최종 패널은 원 기준으로 평가한다. 생성된 후보가 어떤 수정 과제를 만났는지와 어떤 결정에 영향을 받았는지를 기록한다. 한 오류 레코드를 여러 이력이 만났다면 각 노출은 따로 기록하되 오류 레코드 수는 그대로다.

주 결과는 모든 배정 과제를 분모로 하는 라벨 정확도다. 로컬 annotation coverage 진단은 올바른 라벨과 하나 이상의 완전한 주석 근거 집합 포함을 요구한다. 이 정의에서는 전체 문장 번호를 모두 내도 근거 포함 조건을 통과할 수 있다. 따라서 이를 근거 선택의 정밀도나 공식 SciFact 점수로 부르면 안 된다. best-annotated precision은 별도 진단이고, 근거를 늘려 커버리지를 얻는 것과 간결하고 정확한 근거를 고르는 것을 구분하는 데 쓴다.

근거 문장에는 대안 집합이 있을 수 있다. 두 개의 다른 문장 조합이 동일 주장을 충분히 지원할 때 하나만 정답으로 고정하면 유효한 대안을 잘못 거절할 수 있다. 평가기 교정 검토에서 이 대안을 챌린지해야 한다. 주석이 없다는 사실의 해석도 공급된 문서의 범위로 제한한다. 사람이 보기에 모호하

거나 주석이 부족한 사례는 추가 판정 대상이며, 기존 공개 annotation을 무조건 세계의 진실로 선언하지 않는다.

라이선스는 데이터와 코드의 구성을 나누어 기록한다. 확인한 공식 구성은 주장·근거 CC-BY 4.0, 초록 ODC-By 1.0, 코드 Apache 2.0이다. 이는 확인한 공지의 보고이며 법률 인증이 아니다. 공식 라이선스 (https://github.com/allenai/scifact/blob/master/LICENSE.md), 공식 데이터 스키마 (https://github.com/allenai/scifact/blob/master/doc/data.md). 공개 데이터 라벨의 사전학습 노출은 미확인이고, 로컬 evaluator_only 파일로 나눴다고 이미 알려진 정답이 비밀이 되지는 않는다.

10. 독립 자격화와 최종 채점: 유용한 탈락을 결과로 정의하지 않기

유용한 탈락 후보의 정의는 정책의 최종 성공 여부와 분리한다. 후보가 V_old에서 실제로 탈락했다는 조건은 역사 기록에서 확인한다. 유용함은 별도의 교정 자격화 과제에서 확인한다. 예를 들어 원 요구에 맞는 규칙을 실행하고, 사전에 정한 비교 여유와 유지 조건을 충족하는 후보를 자격화할 수 있다. 정확한 수치 기준은 최종 프로토콜에서 잠가야 하며, 아직 완료된 사전등록으로 쓰지 않는다.

이 자격화에 과거 재채점 결과만 쓰면 우연히 맞은 답안을 유용한 능력으로 착각할 수 있다. 최종 정책이 성공한 후보만 유용하다고 정의하면 정책 성공과 자격화가 순환한다. 독립 관측을 사용하는 이유는 "정말 검토할 가치가 있는 탈락 후보가 있었는가"와 "정책이 그것을 찾아 현재 능력을 회복했는가"를 따로 묻기 위해서다. 정책은 숨은 유용함 표시나 최종 정답을 받지 않는다.

정보 누출은 정답 파일에만 있는 문제가 아니다. 보관 후보의 ID가 useful_rejected이고 설명에 "올바른 경계 처리"라고 적혀 있으면, 정책은 평가 없이 정답 후보를 추측할 수 있다. 공개 패킷에서는 c001 같은 중립 ID와 중립 파일명을 쓰고, 실제 수용·거절 결정은 기록된 사실로만 제시한다. 내부 후보 역할, hidden qualification, expected 출력, 채점 코드와 감사 결과는 정책 문맥에서 제외한다. 작성자용 원본 폴더 이름을 그대로 패킷에 복사하지 않는다.

원본 스킬의 파일 해시와 중립 공개 텍스트의 해시도 나누어야 한다. 제목을 가리거나 메타데이터를 정리하면 공개 텍스트는 원본과 다른 바이트가 된다. 둘을 같은 것으로 표시하면 provenance 검증이 깨진다. 평가자는 중립 후보 ID가 어떤 원본 버전을 나타내는지 내부 인덱스로 연결하고, 정책은 숨은 역할을 모르게 한다. 이렇게 해도 스킬의 실제 행동 내용은 공개되므로 과제 요구를 이해해 유용한 전략을 판단하는 것은 허용된 정책 능력이다.

후보 수와 능력 수가 다른 문제도 중요하다. 같은 단위 변환 능력을 제공하는 탈락 후보가 세 개 있고 평균 처리 후보가 한 개라면, 유용 후보 분모는 네 개일 수 있지만 회복해야 할 능력 분모는 두 개다. 정책이 단위 변환 후보 한 개와 새로운 평균 패치로 두 능력을 모두 회복했다면 현재 회복은 성공할 수 있다. 이것을 탈락 후보 네 개를 모두 통합한 것으로 기록하면 잘못이다. 후보→능력 매핑을 감사자 측에 명시한다.

유용한 탈락 후보가 없는 이력의 조건부 recall과 회복률은 NA다. 그런 이력을 지우면 어려운 사례와 적용 불가 범위가 사라진다. 전체 배정 이력의 현재 성공률, 실패와 보류 분모에는 계속 남겨야 한다. 자격화를 수행하지 못한 경우도 "유용한 후보가 없음"과 구분한다. 전자는 분모 미확인이고 후자는 독립 검토 결과 분모가 0인 상태다.

현재 v3 fixture에서는 한 family에 한 유용 탈락과 한 능력을 대응시켜 네 개 분모를 만든다. 이 단순화는 일반적인 다대일 관계를 구현했다고 보기 어렵다. v5 계약은 실제 연구에서 그 매핑을 분리하도록 요구한다. 또한 자격화 입력과 현재 채점 입력, 회복 이후 재발 입력은 서로 다른 관측이어야 한다. 이름만 qualification/test로 나눠 같은 답안과 입력을 재사용하면 독립 평가라고 할 수 없다.

11. 대조군과 요인 실험: 선택 복구와 교훈 복구를 각각 켜 보기

평가기만 교정하는 군은 현재 스킬과 교훈을 그대로 둔다. 현재 승자 수정 군은 활성 스킬을 교정 개발 피드백으로 재작성한다. 무작위 재생 군은 공통 보관함을 같은 시드 규칙으로 탐색하고, 점수 여유 군은 사전에 정한 과거 비교 여유 순서로 탐색한다. 내용 계보 군은 내용 작성 관계를 사용하고, 결정 계보 군은 무효화된 선택·교훈 지원 관계를 사용한다. 이 대조군들은 서로 다른 복구 비용과 성공 양상을 보일 수 있다.

교훈만의 효과를 보는 lesson-frozen 비교는 선택 알고리즘과 후보 풀이 같다는 조건보다 강하다. 확률적인 선택자를 두 번 실행하면 같은 알고리즘에서도 서로 다른 보수 결과가 나올 수 있기 때문이다. 먼저 공통 단계에서 선택·통합한 실제 산출물을 한 번 확정하고 후보 내용, 통합 artifact, 활성 스킬·라우팅, 후속 유효 변경과 교훈 개입 전 상태의 해시를 고정한다. 그 동일 상태를 격리된 두 사본으로 fork한 뒤 외부 교훈만 유지하거나 범위 수정한다. 초기 독립 감사 전에는 선택·통합을 다시 실행하지 않는다. 시작 상태와 스킬·통합 해시는 같고, 교훈 버전이 처치이므로 개입 후 전체 상태 해시

는 달라질 수 있다. 공통 단계의 비용을 양쪽에 동일하게 귀속하고 동일한 추가 예산 상한을 주되, 교훈 수정·검증·실패와 미사용 자원은 따로 기록한다. 이후 지속 진화에서는 교훈 때문에 새 제안과 선택이 달라지는 것이 관측 결과이며 별도 재발 추정량이다.

이 고정 산출물의 짝지은 기억 비교와 전체 2×2 절차 효과를 구분한다. 아래 네 셀은 선택 복구와 교훈 복구 절차를 켜고 끄므로 서로 다른 선택·통합 산출물이 나올 수 있다. 그 상호작용은 전체 절차의 효과이고, 산출물을 같게 유지한 순수 교훈 대비와 같지 않다. 선택은 그대로 두고 교훈만 고치는 셀도 필요하며, v3의 일곱 고정 행동 계획이 모든 요인 셀이나 실제 paired memory fork를 완성했다고 쓰지 않는다.

네 셀은 선택 복구 꺼짐/켜짐과 교훈 복구 꺼짐/켜짐이다. 모든 셀의 평가기는 V_new다. Y00은 두 복구를 하지 않은 결과, Y10은 선택만 복구한 결과, Y01은 교훈만 복구한 결과, Y11은 둘을 복구한 결과다. 관심 상호작용은 Y11−Y10−Y01+Y00이다. 이 식은 두 개입을 함께 할 때의 추가 차이를 설명하며, 어떤 숨은 원인이 존재하지 않는다는 증명은 아니다.

교육용 수치를 넣으면 이해가 쉽다. 현재 성공률이 Y00=0.30, Y10=0.60, Y01=0.40, Y11=0.80이라면 상호작용은 0.10이다. 선택 복구의 차이는 교훈을 고치지 않은 조건에서 0.30, 교훈을 고친 조건에서 0.40이다. 이 차이가 반복해서 관측된다면 두 개입이 보완적으로 작동한다는 가설을 검토할 수 있다. 그러나 이 숫자는 가상의 계산 예시이며 실제 측정 효과나 유의성은 아니다.

상호작용이 0이라고 해서 교훈이 아무 역할도 하지 않았다는 뜻은 아니다. 두 개입이 독립적으로 비슷한 효과를 낼 수도 있고, 표본이 부족해 차이를 발견하지 못할 수도 있다. 반대로 양의 상호작용만으로 잘못된 교훈이 원래 오류의 유일한 원인이었다고 결론 내리지 않는다. 정책 절차가 함께 변하거나 예산이 달라지면 같은 패턴이 나올 수 있다. 패킷·자원·실행 순서·개입 내용이 정확히 일치하는지 먼저 점검한다.

인과 해석의 단위는 정책 배정 전 동결된 실제 진화 이력이다. 같은 이력을 정책별로 복제하여 짝지은 결과를 얻고, 실행 순서와 난수 설정을 통제하거나 무작위화한다. 정책 간 메모리와 도구 상태가 섞이지 않아야 하며, "교훈을 고친다"는 처치가 매 실행에 동일한 정의로 구현되어야 한다. 이 조건에서 절차 효과를 추정할 수 있어도 모든 모델과 모든 장기 환경에 대한 인과 법칙은 주장할 수 없다.

허위 교정 또는 sham 대조군도 의미가 있다. 원래 맞던 평가기에서 수정이 필요 없다는 알림을 주고 동일한 추가 검토 절차와 비용 기회를 제공한다. 만약 그 군도 비슷하게 좋아진다면 교정된 근거의 수선보다 추가 시간과 탐색 자체가 효과를 냈을 수 있다. 잘못된 시작점, 누락된 연결, 일부 보관함만 남긴 상황도 감도 분석으로 시험한다. 이들은 좋은 결과를 만들기 위한 조건이 아니라 제안 기제가 의존하는 전제를 드러내는 대조다.

12. 같은 예산과 정보: 선택자 품질과 제안자 품질을 나누기

공통 보관함이라는 말은 모든 정책이 정확히 같은 후보 내용을 접근할 수 있다는 뜻이다. 결정 계보 정책만 좋은 후보를 미리 받거나 무작위 정책만 후보 수가 많고 긴 설명을 받으면 비교가 성립하지 않는다. 후보 파일과 공개 메타데이터의 스냅샷, 접근 가능한 개발 예제, 교정 알림, 점수 피드백 권한을 먼저 맞춘다. 정책이 정보 구조를 다르게 사용하는 것은 허용되지만 숨은 정답 접근권이 달라지는 것은 허용하지 않는다.

내용 계보 군은 의도적으로 결정·교훈 연결을 사용하지 않는다. 이 제거는 비교하려는 정보 개입이며 문서에 명시한다. 두 정책이 받는 데이터의 크기가 달라질 수 있으므로 그 차이와 접근 비용을 기록한다. "정보가 완전히 동일하다"는 표현보다 같은 후보·공개 과제·개발 피드백 권한을 가지고 어느 계보 신호를 사용할지가 실험 요인이라고 쓰는 편이 정확하다. 최종 감사의 gold는 모든 비오라클 정책에 동일하게 숨긴다.

먼저 후보 풀을 고정하면 정책이 무엇을 고르는지 평가할 수 있다. 모든 정책이 같은 실제 archive에서 시작하고 새 후보를 작성하지 않는 선택 단계에서는 후보 품질 차이를 정책의 제안 능력으로 설명할 수 없다. 회복 속도나 선택 recall 차이는 탐색·순서·판정·통합의 차이와 연결된다. 이때도 현재 승자 수정이 허용되는 군은 후보 생성 비용이 붙는 별도 강한 대조로 해석해야 한다.

지속 진화 단계에서는 새 후보를 만들 수 있으므로 정책별 후보 풀이 달라진다. 이 단계의 결과는 선택자와 제안자의 효과가 섞인 새 estimand다. 동일 초기 checkpoint와 같은 예산으로 시작해도 이후 생성 경로가 같다는 보장은 없다. 첫 단계의 고정 풀 결과와 두 번째 단계의 재발·제안 결과를 별도 표와 별도 분모로 보고하면 어느 구성요소가 도움을 주었는지 더 명확해진다.

예산은 사후 교정 행동만 세는 것으로 충분하지 않다. 공통 역사 생성과 보관함 제작 비용, 숨은 자격화·감사 비용은 정책 비용과 구분해 공개해야 한다. 정책별로는 후보 읽기, 검색, 재채점, 새 모델 실행, 도구 호출, 수정 작성, 현재 통합, 교훈 버전 작성과 재검증 비용이 필요하다. 동일 최대 상한을 주

어도 실제 지출이 다르면 사용하지 않은 예산을 남긴다. 값싼 군이 같은 성능을 내는 것도 의미 있는 결과다.

v3의 24 단위 상한과 재채점 1·새 실행 2·통합 1·수정 템플릿 준비 3·교훈 수정 1은 합성 행동 비용이다. 토큰이나 원화·달러 비용으로 변환할 근거가 없다. 실제 모델 연구에서는 호출 로그에서 입력·출력 토큰과 과금 정보, 지연을 따로 수집해야 한다. 도구 실행 시간이나 감사 비용까지 포함할 것인지도 프로토콜에서 정한다. 자원이 측정되지 않았으면 unknown으로 남긴다.

예산이 떨어졌을 때는 마지막으로 확정된 상태를 유지한다. 다음 행동을 시도할 수 없었다는 보류 기록과 남은 후보를 남긴다. 평가하지 않은 수정본을 활성화하거나 보류를 성공으로 채우면 제한 예산 연구의 의미가 사라진다. 정책이 과거 답만 재채점해서 값싸게 높은 점수를 얻었더라도 현재 실행을 하지 않았다면 현재 회복 결과는 아직 미확인이다. 그래서 예산 기록은 세 endpoint와 함께 읽어야 한다.

13. 세 endpoint와 실제 응답 입력: 같은 답안에 새 이름을 붙이면 안 된다

저장 답안 재채점은 오래된 응답 바이트를 그대로 두고 V_new로 점수를 매기는 작업이다. 원 답안 해시가 유지되므로 모델 호출 없이도 할 수 있다. 옛 평가가 잘못된 답을 좋아했는지, 좋은 답을 거절했는지 확인하는 데 유용하다. 그러나 입력 시각을 오늘로 바꾸거나 endpoint 이름을 fresh로 적는다고 새로운 모델 실행이 되지 않는다. 재채점 receipt에는 과거 답안의 부모 기록과 원 해시를 명시한다.

새 후보 재실행은 현재의 후보 파일과 고정 입력을 실제 runner가 새로 모델에 넣어 얻은 응답이다. 모델 ID와 설정, 입력 패킷·후보·과제 해시, 실행 시각, 응답 바이트, 도구 호출 및 자원 로그가 연결되어야 한다. 과거 답과 우연히 같은 문자열이 나올 수도 있지만 새 invocation 기록이 있으면 새 실행이다. 반대로 문자열이 새로 보이더라도 생성 근거가 없으면 외부에서 받은 아티팩트일 뿐이다. 텍스트의 모양만으로 신선성을 판단하지 않는다.

현재 상태 회복은 그 후보가 실제로 활성화된 상태에서 독립 입력을 다시 실행하는 결과다. 후보를 파일로 저장한 것과 활성 포인터를 바꾼 것, 실제 문맥에 들어간 것은 각각 다르다. 활성 스킬 버전과 외부 교훈, 라우팅 또는 조립 규칙, 후속 유효 변경이 어떤 상태로 유지되었는지 기록해야 한다. 정책의 개발 과제를 그대로 최종 감사에 사용하면 수정 과정에 적응한 결과를 독립 회복으로 착각할 수 있다.

실제 모델이 한 후보를 잘 실행했는데 현재 시스템에서 다른 스킬이 먼저 활성화되면 최종 상태는 실패할 수 있다. 다른 후보로 회복하거나 새 패치로 회복한 경우도 인정하되, 역사적 탈락 후보를 통합한 것으로 기록하지 않는다. "현재 능력 회복"은 목표 능력을 다시 수행했다는 평가 결과이고 "특정 후보 채택"은 상태 변경 사실이다. 두 필드를 유지하면 회복 경로를 정확히 설명할 수 있다.

v3의 prepare-trials는 중립 후보 ID, 실제 SKILL 텍스트, 공개 입력과 계약, 동일 공개 archive 및 교정 안내로 정책 패킷을 만든다. 감사 패킷과 내부 매핑은 정책 입력에서 분리한다. 전체 패킷 폴더를 에이전트에 넘기면 내부 스냅샷을 읽을 수 있으므로, 개별 agent packet만 전달해야 한다. 이 exporter는 운영체제 권한을 강제하지 않는다. 실제 실행에서 숨은 evaluator 자료를 읽지 못하게 하는 별도 접근 통제가 필요하다.

v3 import는 제공받은 한 응답을 해당 과제의 grader에 넣고 receipt를 남긴다. 요청한 endpoint, 선언한 모델 ID·시각·부모·state hash와 controller가 받은 시각을 구분한다. 입력 파일과 패킷이 맞는지는 검사할 수 있지만, 선언한 모델이 실제로 실행되었는지나 그 state가 실제로 활성화되었는지는 인증하지 않는다. 그래서 fresh_model_execution_verified와 current_recovery_verified는 false, verified_model_id는 null, 과학적 실험 상태는 NR로 유지한다.

실제 성능표에 넣으려면 import 주변에 신뢰 가능한 invocation·activation 증거가 추가되어야 한다. 외부 실행자를 절대적으로 신뢰한다는 선언만으로 충분하지 않고, 무엇을 확인했고 무엇은 선언에 의존하는지 나누어야 한다. 실제 runner가 저장한 task·packet·candidate·state 연결과 독립 감사가 있어야 한다. 과거 응답을 재사용한 실험이라면 그 사실 자체를 자료로 남기고 재채점 endpoint로 보고한다.

로컬 manifest와 semantic recompute는 다른 범위의 검증이다. raw 응답과 계산 지표가 일치하고 파일이 빠지거나 바뀌지 않았는지 확인한다. manifest를 다시 해시하여 점수만 바꾼 변조도 의미 재계산으로 거절할 수 있다. 그러나 신뢰 기준 코드와 모든 원본을 동시에 다시 쓴 공격의 외부 진실성을 보장하지는 않는다. 무결성 검증, 실제 실행 provenance, 학술적 타당성은 각각의 주장이다.

14. 정량 평가 I: 왼쪽 계단 RAUC와 최종 성공률

복구의 좋은 점이 마지막 결과에만 있지는 않다. 같은 예산 상한에서 둘 다 성공해도 한 정책이 더 빨리 회복하면 실용적 차이가 있다. 선언한 최대 복구 예산을 B라 하고 관측 비율을 0=q0<q1<…<qJ=1로 정한다. 각 지점의 비용은 b_j=q_jB다. 그 시점까지 실제 비용 이하에서 확정된 가장 최근 상태를 독립 과제로 평가하여 U_j를 얻는다. 미래에 성공할 수정본을 앞선 checkpoint에 가져오지 않는다.

주 면적 요약은 왼쪽 계단 방식이다. 각 구간에서 구간 시작의 관측 성공률을 사용한다. 오른쪽 끝에서 처음 좋아진 결과를 앞 구간에 미리 배분하지 않는 정의다. 그 때문에 사다리꼴 보간으로 관측하지 않은 회복을 가정하는 방식과 다르다. 다만 구간 중간에 실제 상태가 바뀌었을 수 있으므로, 이 값은 선언한 관측 격자의 요약이지 전체 시간의 정확한 적분은 아니다.

$$b_j = q_j B$$

$$\mathrm{RAUC} = \sum_{j=0}^{J-1} (q_{j+1} - q_j) U_j$$

$$\mathrm{FinalSuccess} = U_J$$

교육용으로 q=[0,0.25,0.50,0.75,1.00]이고 U=[0,0,0.5,1,1]이라고 하자. RAUC는 0.25×0+0.25×0+0.25×0.5+0.25×1=0.375다. 마지막 성공률은 1이다. 동일 예산 상한의 다른 가상 정책 U=[0,0.5,1,1,1]은 RAUC=0.625, 마지막 성공률은 역시 1이다. 이 숫자는 정의를 설명하는 직접 계산 예시이며 실제 정책 우열의 측정값이 아니다.

마지막 시점에서만 성공한 U=[0,0,0,0,1]의 RAUC는 0이다. 마지막 값 이후에 폭이 있는 구간이 없으므로 면적에 들어가지 않기 때문이다. 그 정책의 마지막 성공률은 1로 따로 보고한다. 이 사례는 RAUC만 보고 완전히 실패했다고 쓰면 안 되는 이유를 보여 준다. 반대로 초반 성공 후 마지막에 실패한 정책은 높은 RAUC와 낮은 최종 성공률을 가질 수 있다. 회복의 지속성과 재발은 별도 지표가 필요하다.

결측 처리도 식에 맞아야 한다. U=[0,None,1,1,1]이라면 q=0.25에서 시작하는 양의 폭 구간의 왼쪽 값이 없으므로 RAUC는 NA다. 빈 값을 0이나 이전 성공으로 몰래 채우면 안 된다. U=[0,0,1,1,None]이라면 관측된 왼쪽 네 값으로 RAUC=0.5를 계산할 수 있지만 최종 성공률은 NA다. 마지막 결측은 앞 구간의 값과 마지막 endpoint를 동시에 지우는 것으로 처리하지 않는다.

잘못된 답이나 명시적 실패·보류 응답은 관측된 실행 결과이며 전체 배정 과제의 실패로 셀 수 있다. checkpoint 자체를 실행하지 못했거나 receipt를 잃은 경우는 관측 부재다. 두 경우를 구분하지 않으면 실제 정책 실패와 측정 시스템의 누락을 혼동한다. RAUC의 checkpoint가 존재하더라도 일부 과제에 결과가 없으면 해당 주 성공률의 완전성이 충족되는지 확인하고 결측으로 표시해야 한다.

예산 지점은 요청 횟수인지 실제 비용인지 정해야 한다. 토큰 예산이 주 기준이면 매 행동의 실제 사용량으로 b_j 이하의 확정 상태를 선택한다. "단계 2를 마쳤다"는 이름만 비용 50%에 대응시킬 수 없다. 공통 감사 비용을 주 예산에 포함할 것인지 별도 보고할지도 고정한다. 통계 비교에 쓰는 RAUC는 모든 정책에서 같은 격자·분모·결측 규칙을 사용하며 사후에 유리한 관측 지점으로 바꾸지 않는다.

15. 정량 평가 II: 현재 회복, 첫 관측 도달, 재발

유용 탈락 검토 recall은 독립 자격화된 탈락 후보 중 실제로 검토한 고유 후보 수를 분자로 한다. 전체 유용 탈락 후보 수가 분모다. 재채점을 한 후보와 새 실행까지 한 후보, 실제로 통합한 후보 수는 다를 수 있으므로 단계별 기록이 필요하다. recall이 높아도 통합이 실패하거나 활성 교훈이 방해하면 현재 회복은 낮을 수 있다. 탐색 성과를 최종 능력으로 바꾸어 부르지 않는다.

현재 검증 회복률은 독립 자격화된 목표 능력 중 현재 상태가 독립 새 입력에서 수행한 능력 수를 분자로 한다. 교육용으로 유용 후보 여섯 개가 세 능력을 나타내고 정책이 세 후보를 검토하여 두 능력을 회복했다면 후보 recall은 3/6=0.5, 능력 회복률은 2/3이다. 이 사례는 가상 매핑이다. 하나의 후보가 여러 능력을 담당하는 경우도 있어 매핑과 채점 규칙을 사전에 정해야 한다. 빈 능력 분모는 NA다.

회복까지의 시간은 사전에 정한 목표 θ를 처음 관측으로 만족한 checkpoint의 예산이다. θ를 결과를 본 뒤 낮추면 정책이 더 빨리 성공한 것처럼 보일 수 있다. 첫 관측 hit 뒤에 다시 실패하더라도 첫 hit 자체를 지우지 않는다. 첫 도달, 최종 성공, 유지 성공, 재발은 서로 다른 질문이다. 비단조적인 곡선을 처음부터 단조 증가했다고 가정하면 잘못된 보간과 검열 해석이 생긴다.

교육용 U=[0,None,1,1,1], q=[0,0.25,0.5,0.75,1], θ=1을 보자. 첫 관측 hit는 q=0.5다. q=0.25가 결측이라는 사실도 함께 남겨야 한다. observation_complete=false, missing_checkpoints=[0.25], incomplete_window=true로 표시한다. 처음 hit 이전까지도 결측이 있으므로 hit_window_complete=false다. 이 정보를 누락하고 q=0.5만 보고하면 모든 앞 구간에서 실패를 확인한 것처럼 오해할 수 있다.

마지막 관측 실패 q=0에서 첫 관측 hit q=0.5로 이어지는 관측 전환 구간은 기록할 수 있다. 하지만 진정한 최초 회복 시각이 반드시 그 안에 있다는 보장은 없다. 관측 사이에 먼저 성공했다가 다시 실패하고 이후 또 성공했을 수도 있고, 이미 앞선 결측 지점에서 성공했을 수도 있다. 특히 비단조 곡선에서는 마지막 관측 실패 이후의 구간을 첫-ever 회복의 엄밀한 상하한으로 부르면 안 된다. 단조성 같은 추가 가정이 없다면 "관측된 실패→hit 구간"이라고만 쓴다.

목표에 한 번도 관측 도달하지 못한 이력은 검열한다. 예산 상한 B를 성공 시각으로 넣지 않는다. 계획된 검열 상한과 마지막 실제 관측 예산을 따로 보존한다. 교육용 U=[0,0,None,None,None]은 q=0.25까지만 실패를 관측한 것이며, q=1까지 완전 관측한 실패와 같지 않다. 결측과 미도달이 섞인 사례의 시간 분석은 표본 크기와 가정이 준비되기 전까지 단순 receipt와 상태로 보고하는 편이 정확하다.

재발의 조건부 정의는 별도 초기 회복 자격화를 통과한 이력이 나중 독립 패널에서 실패하는 것이다. 분모는 초기 회복이 확인된 이력 수이고 그 수가 0이면 NA다. 전체 이력에서는 이후 패널 실패율을 함께 보고할 수 있지만, 처음부터 회복하지 못한 이력의 실패를 엄밀한 재발이라고 부르면 안 된다. 전체 보고에 initial_recovered, later_failed, missing 상태를 남겨 조건부 재발과 전체 이후 실패를 구분한다. 필요한 경우 초기 회복과 이후 실패를 동시에 만족하는 이력 비중도 별도 표기한다.

v3는 활성 교훈이 남으면 이전 승자로 돌아가도록 프로그램한 재발 fixture를 갖고 있다. 그것은 실제 모델이 같은 오류를 자연스럽게 재발시켰다는 증거가 아니다. 실제 재발 연구에는 새 제안 생성, 선택, 교훈 검색과 노출, 이후 입력의 독립성이 필요하다. lesson-frozen과 lesson-repaired가 같은 후보와 예산을 사용한 상태에서 지속 진화를 관측해야 교훈 수정의 재발 억제 효과를 검토할 수 있다.

16. 정량 평가 III: 손상 분모, 짝지은 비교와 불확실성

회복 정책이 한 능력을 고치면서 다른 기능을 망가뜨리면 단순 성공률만으로는 충분하지 않다. 영향받지 않아야 할 과제 집합 H를 따로 정한다. H의 소속은 그래프에 연결이 없다는 사실만으로 결정하지 않는다. 교정된 규칙의 영향을 받는 입력인지 계약을 검토하고, 원 현재 상태가 별도 자격화 입력에서 성공하는지도 확인한다. 자격화 입력과 최종 손상 평가 입력은 다른 관측으로 유지한다.

H에서의 절대 손상 실패율은 1에서 새 상태의 평균 성공률을 뺀 값이다. 추가 손실은 원 상태의 성공률에서 새 상태 성공률을 뺀 평균이다. 두 지표는 기준 상태가 완벽하지 않을 때 특히 달라진다. 새 상태가 이전보다 좋아지면 추가 손실은 음수일 수 있다. 음수를 0으로 잘라 놓으면 개선 정보를 지우며, 절대 실패가 남았다는 사실도 숨길 수 있다.

$$F_H(S) = 1 - \frac{1}{|H|}\sum_{t\in H} p_t(S)$$

$$R_{\text{excess}}(S) = \frac{1}{|H|}\sum_{t\in H}(p_t(S_0) - p_t(S))$$

교육용 평균을 넣어 보면 기준 성공률 0.70, 새 성공률 0.90일 때 절대 실패는 0.10, 추가 손실은 -0.20이다. 기준 성공률 1.00, 새 성공률 0.90인 다른 사례는 절대 실패 0.10이 같지만 추가 손실은 0.10이다. 이 계산은 가상 설명이며 실제 손상 결과가 아니다. H가 비어 있거나 자격화를 수행하지 못했다면 두 지표는 NA로 남기고 분모 상태를 기록한다.

주 비교의 표본은 동결된 진화 이력이다. 하나의 이력에서 여러 정책이 실행되면 정책 차이를 짝지어 비교할 수 있다. 같은 이력의 다섯 checkpoint를 다섯 독립 표본으로 세면 안 된다. 동일 모델 세션, 같은 초깃값 스킬, 공유 문서 성분도 결과 의존성을 만들 수 있다. 이력 단위의 집계와 과제 성분·세션 수준의 계층 구조를 구분하고, 실제 수집 수가 확인된 다음 적절한 분석을 정한다.

531개 과제와 325개 성분이라는 준비 수는 표본 설계의 자원이다. 그것만으로 효과 검출력이 충분하다고 계산할 수 없다. 실제 오류 노출 빈도, 유용 탈락 비율, 교훈 작성률, 정책 간 차이의 분산과 상관이 아직 없다. 임의의 표본 수나 좁은 신뢰구간을 적지 않는다. 파일럿에서는 이런 빈도와 분산을 추정하고, 최소 의미 효과와 최종 이력 수를 고정한 새 프로토콜로 이동할 수 있다. 그때도 파일럿과 최종 결과를 분리한다.

반복 응답 성공률과 pass@k는 같은 지표가 아니다. 같은 과제에서 k번 실행해 성공한 비율은 반복 성공률이다. pass@k는 k개 중 적어도 하나가 성공할 가능성에 관한 질문이다. n개 표본 중 c개가 성공했을 때 조합 기반 추정식 $1-C(n-c,k)/C(n,k)$을 사용할 수 있지만, n·c·k와 표본 생성 조건을 명시해야 한다. 한 번의 현재 상태 실행이 필요한 과제에서는 best-of-k 결과를 단일 실행 회복률로 보고하지 않는다. 모든 k개가 성공하는 all@k를 선택하더라도 별도 정의다.

여러 정책·task family·지표를 동시에 보고 가장 유리한 차이만 주 결론으로 고르면 우연을 기제 효과로 오해할 수 있다. 최종 시험 전에 주 contrast, 주 endpoint, 최소 의미 차이, 손상 허용 기준, 결측 처리, 검정 또는 구간 추정, 다중 비교 처리를 정한다. 현재 계약은 그 필요성을 적고 있을 뿐 완료된 통계 계획이 아니다. 결과가 없으므로 p값·효과 크기·정밀도를 생성하지 않는다.

부정적인 결과도 분모를 갖는다. 유용 탈락이 없는 이력의 수, 교훈이 실제로 노출되지 않은 수, 예산 때문에 미완료인 수, 파싱 실패와 실행 실패, 현재 회복 없이 재채점만 성공한 수를 보고하면 어디에서 현상이 성립하지 않았는지 알 수 있다. 이런 기록이 있어야 잘된 몇 사례로 전체 방법의 성능을 대표하지 않을 수 있다.

17. 반증 조건과 첫 주 결정: 어떤 관측이 설계를 바꿔야 하는가

첫 반증 지점은 평가기 교정의 타당성이다. 새 평가기가 경계값이나 대안 정답을 다시 잘못 거절한다면, 뒤의 정책 비교는 아직 교정된 기준을 사용하지 못한 것이다. 사람 검토가 필요한 사례를 강제로 라벨링하거나 요구를 바꾸어 결과를 맞추면 기본 연구 질문이 무너진다. 독립 challenge에서 불확실한 사례는 보류하고 G*의 범위와 허용 답안을 다시 정리해야 한다.

둘째는 현상의 관측 가능성이다. 실제 모델이 생성한 이력에서 유용한 탈락 후보가 거의 없거나 부정적 교훈이 생성·노출되지 않을 수 있다. 그런 결과는 제안 방법이 필요한 상황이 드물다는 증거다. 합성 후보와 교훈을 주입해서 항상 발생하게 만들면 실험 도구의 반응은 검증할 수 있지만 자연 발생 빈도는 알 수 없다. 빈 이력과 적용 불가 이력을 삭제하지 않고 파일럿의 중요한 결과로 보존한다.

셋째는 단순 대조군으로 충분한지다. 교정된 평가기만 사용해도 현재 상태가 잘 작동하거나, 현재 승자 수정이 같은 비용에서 더 빨리 회복한다면, 결정 이력 복구의 추가 복잡성을 정당화하기 어렵다. 보관 후보 무작위 재생이 같은 성능을 낸다면 계보 기반 선택의 이점도 약하다. "어떤 방식이든 과거를 다시 보니 좋아졌다"는 관측을 제안된 결정·교훈 기제의 증거로 바꾸지 않는다.

넷째는 교훈 개입의 독립 효과다. 선택을 같게 유지한 lesson-frozen과 repaired 군이 차이를 보이지 않으면 그 설정에서 교훈 수정의 추가 이득이 확인되지 않은 것이다. 표본 부족과 실제 무효를 구분하려면 사전 효과 기준과 불확실성이 필요하다. 교훈을 고친 군이 다른 능력을 더 손상한다면, 한 패널의 성공으로 안전한 회복을 주장하지 않는다. 범위를 좁힌 교훈 수정이 필요한지 또는 이 방법을 보류할지 결정한다.

다섯째는 계보와 비용 가정이다. 기록 연결을 일부 삭제하거나 잘못된 교정 시작점을 넣었을 때 정책이 실패한다면 그 감도와 실제 로깅 요구를 명시한다. 모든 후보를 읽고 숨은 자격화를 수행하는 비용을 빼야만 이점이 생기면 실용적 우위는 아직 없다. 외부 source manifest가 완전하다는 사실도 모델의 실질적 문맥 의존성을 모두 기록했다는 증거가 아니므로 그래프의 완전성 주장은 좁게 유지한다.

첫 주에는 순서가 있는 중단 조건이 필요하다. C0에서는 요구·대안 정답·교정 challenge를 고정한다. C1에서는 패킷의 정답 누출과 실행 provenance를 검토하고 모델·전송 범위·예산을 정한다. C2에서는 실제 V_old 진화 이력을 수집한다. C3에서는 후보와 능력을 독립 자격화한다. C4에서는 같은 상태의 정책 forks와 요인 비교를 수행한다. C5에서는 손상과 이후 재발 패널을 평가하고 C6에서 최종 분석 계획과 버전을 잠근다. 지금은 이 checkpoint의 설계가 있고 실제 실행은 아직 없다.

이 순서는 "첫 주에 모든 실험을 완료한다"는 약속이 아니다. 모델 설정과 권한이 없으면 C1에서 실제 호출이 대기할 수 있다. 교정이 모호하면 C0의 기준 검토가 먼저다. 유용한 탈락이 없는 파일럿이면 C2–C3의 빈 분모를 결과로 남기고, 다른 과제나 노출 조건을 쓰는 새 설계를 명시적으로 작성한다. 사후 변경은 새로운 버전으로 기록하며 기존 자료의 상태를 바꾸지 않는다.

최종 원고에서 인정할 수 있는 깊은 통찰은 두 가지다. 교정은 점수 수정에서 끝나지 않고 판단을 지원한 외부 상태의 유효성까지 문제를 만든다는 것, 그리고 그 지원 경로를 알아도 현재 능력의 회복이 자동으로 따라오지 않는다는 것이다. 첫 통찰은 관측할 인과 경로를 제안하고, 둘째는 과장 없는 endpoint와 실패 대조군을 요구한다. 지금은 검증 가능한 설계의 통찰이며 실증된 효과가 아니다.

18. 논문 페이지와 해설서 연결, 자주 묻는 질문, 남은 불확실성

4페이지 영문 원고는 이 긴 설명의 주장 구조를 압축한다. 1페이지는 고정 요구와 오염된 현재 상태의 문제, 기존 방법과의 차이, 미실측 경계를 소개한다. 해설서 1·2·6·7절이 그 배경이다. 2페이지는 고정 답안 순위 식과 기록 경로 충분조건, 실제 이력 fork의 개입 설계이며 3·4·5·11절과 연결된다. 3페이지는 준비된 과제·비교군·endpoint·RAUC이며 8·9·10·12·13·14절이 이를 풀어 설명한다. 4페이지는 빈 분모·검열·손상·반증·다음 단계이며 15·16·17절을 참고하면 된다.

질문 1. 평가기를 고쳤는데 왜 스킬도 다시 손봐야 하는가? 과거 평가가 현재 승자와 외부 교훈을 이미 선택했기 때문이다. 새 평가기를 설치하는 것은 앞으로의 점수를 바꾼다. 현재 상태가 그대로라면 새 입력에서 같은 오류가 남을 수 있다. 그런 효과가 실제로 있는지는 grader-only와 상태 복구 군의 새 실행으로 확인하며, 반드시 손봐야 한다고 전제하지 않는다.

질문 2. 왜 평가 오류를 일부러 넣는가? 고정 요구와 오류의 위치·시점을 알아야 의도한 인과 경로를 시험하기 쉽기 때문이다. seed fault는 운영 오류의 빈도를 추정하는 자료가 아니다. 자연스러운 실제 평가 버그를 독립 검토해 수집하면 외적 타당성이 좋아질 수 있지만 비용과 애매함도 커진다. 두 provenance를 같은 표의 "발견된 오류"로 합치지 않는다.

질문 3. 좋은 탈락 후보를 보관하면 끝나는가? 보관은 접근 가능성을 만든다. 무엇을 먼저 읽고 검증할지, 현재 유효 변경과 어떻게 통합할지, 과거의 금지 교훈이 이후 제안을 다시 막는지를 해결하지 않는다. 그러나 단순 보관 재생으로 충분할 수도 있다. 그래서 같은 archive의 random/margin replay가 중요한 대조다.

질문 4. 계보가 있으면 인과를 알 수 있는가? 내용 작성과 선택·교훈 지원·실제 문맥 노출의 연결은 관측한 관계다. 다른 문맥과 모델 지식이 결과를 함께 만들 수 있으므로 연결만으로 원인을 확정하지 않는다. 같은 이력 fork에서 선택과 교훈을 각각 개입하고 새 결과를 보는 것이 추가 근거다. 그래프의 충분조건은 기록 경로 감사에 한정한다.

질문 5. 해시 검증이 통과하면 실제 실험 결과라고 할 수 있는가? 아니다. 해시는 파일 동일성과 관계를 확인한다. import한 응답의 모델·시각·활성 상태는 별도 실행 증거가 없으면 선언일 뿐이다. raw와 지표 재계산이 일치해도 모델 호출이 없었다면 아티팩트 채점 결과다. 과학적 NR을 성공으로 바꾸지 않는다.

질문 6. 531개 과제와 44개 오류면 논문 실험이 충분한가? 과제와 구성 성분은 준비되어 있지만 실제 후보 이력·교정 이력은 0개다. 선택 비교의 단위는 실제 진화 prefix이고, useful reject와 노출 교훈이 얼마나 생기는지도 모른다. 파일럿이 그 빈도와 비용을 측정한 뒤 최종 표본 계획을 정해야 한다. 44개의 오버레이를 44개의 복구 성공으로 해석할 수 없다.

질문 7. 옛 탈락 후보를 복원하면 나중의 개선이 사라지지 않는가? 그 위험이 있다. 오래된 파일을 무조건 현재 파일로 교체하지 않고 현재 상태의 유효 변경과 통합한 후 새 채점을 한다. 기준 능력과 무영향 H를 별도 확인한다. 보관 후보는 유용한 근거 또는 수정 재료일 수 있으며 전체 rollback을 자동으로 정당화하지 않는다. 현재 v3 라우팅 사전은 이런 일반 통합을 구현한 것이 아니다.

질문 8. 처음 hit를 관측했는데 뒤에 실패하면 회복 시간도 취소되는가? 첫 관측 도달은 유지한다. 최종 성공과 이후 실패를 따로 보고한다. 앞 checkpoint가 빠졌으면 결측도 남기며, 비단조 상태에서 관측 전환 구간을 진짜 최초 회복의 엄밀한 경계로 주장하지 않는다. 여러 지표를 함께 보면 조기 성공과 안정성의 차이가 드러난다.

질문 9. 직접 계산한 예시와 합성 fixture를 왜 넣는가? 분모·수식·상태 변화·변조 거절을 검증하기 위해서다. 교육용 가상 수치, 직접 대수 계산, scripted fixture, 실제 모델 관측은 서로 다른 증거 유형이다. fixture의 active-only와 decision-aware가 같은 결과를 낸다고 해서 실제 모델에서도 같다고 할 수 없고, 다른 fixture가 유리하다고 해서 제안 방법이 우수하다고 할 수도 없다.

질문 10. 현재 가장 큰 불확실성은 무엇인가? 실제 이력의 유용 탈락과 잘못된 교훈 발생률, 계보 수집의 완전성, 교정의 도메인 타당성, 단순 대조 대비 비용 효과, 후속 개선의 유지, 공개 정답의 사전 학습 노출, 표본·통계 계획이다. 모델 ID, 버전 고정 외부 adapter와 사전등록도 미완료다. 이 불확실성을 숨기지 않는 것이 다음 실행을 구체적으로 만든다.

이 해설서와 함께 METHODS_CONTRACT.json에서 개입·분모·결측·중단 조건을 읽고, SOURCES.json에서 공식 출처와 로컬 원본 해시 및 주장 상태를 확인할 수 있다. paper.json은 조판 입력이며 실제 성능을 담은 결과 파일이 아니다. 기존 v3/v4 자료와 활성 스킬을 수정하지 않았고 새 의존성·모델 호출·훈련·데이터 다운로드를 수행하지 않았다. 다음 실제 실행의 provenance는 별도의 새 출력 폴더와 고정된 runner 기록으로 시작해야 한다.